claude-windows / 90c0b6b8-7430-4870-98cb-e487eacfa1d7

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\90c0b6b8-7430-4870-98cb-e487eacfa1d7\subagents\workflows\wf_ad5139d3-ec0\agent-abbcf604d6fe2db43.jsonl`
- SHA-256: `920fa7184678249194b252dfa2f9dbdd0aa298ce5b7f6688a9fa1ec81da0c7aa`
- Source modified: `2026-07-04T01:50:12+00:00`
- Imported at: `2026-07-05T16:48:25+00:00`
- project: `wf_ad5139d3-ec0`
- session_id: `90c0b6b8-7430-4870-98cb-e487eacfa1d7`

Transcript

1. user (2026-07-04T01:48:32.874Z)

```
Repo root: C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer. Read
docs/COMPUTE_DECOUPLED_SERVING/README.md (esp. sections 2, 3, 3a), deploy/cloudrun/README.md,
and docs/ALPHA_LAUNCH_READINESS.md (the A1 row / section 7). Also skim
docs/COMPUTE_DECOUPLED_SERVING/runlogs/ for any existing DEPLOY_REPORT_cloud-serving_*.md.
Answer, each with file/section evidence:
1. The documented deploy procedure for BOTH the control-plane service and the L4 worker job.
List the exact required gcloud flags the docs call out (especially --no-cpu-throttling, --min-
instances, --set-env-vars, GPU flags).
2. What DEPLOYMENT_PROFILE=cloud-serving changes/gates in the code and docs.
3. The A1 acceptance criteria / definition-of-done from the tracker.
4. Documented gotchas about storage permissions (Cloud Run service account needing
roles/storage.objectAdmin on the bucket, ADC) and about the worker image not bundling backend/ui
(SPA served elsewhere).
5. Whether a DEPLOY_REPORT_cloud-serving_2026-07-04.md ALREADY exists in runlogs (from PR #457)
and what it covers, so I don't overwrite it.
Return structured findings + gaps.
```

2. assistant (2026-07-04T01:48:35.955Z)

```
I'll start by pulling the repo per convention, then read the specified docs.
```

3. user (2026-07-04T01:48:41.030Z)

```
Already up to date.
---LS RUNLOGS---
total 20
drwxr-xr-x 1 avidu 197609    0 Jul  4 06:58 .
drwxr-xr-x 1 avidu 197609    0 Jun 25 16:24 ..
-rw-r--r-- 1 avidu 197609 4860 Jul  4 06:58 DEPLOY_REPORT_cloud-serving_2026-07-04.md
-rw-r--r-- 1 avidu 197609 2261 Jun 25 16:24 README.md
-rw-r--r-- 1 avidu 197609 2698 Jun 25 16:24 RUNLOG_truly-local_TEMPLATE.md
```

4. user (2026-07-04T01:48:41.050Z)

```
1       # Cloud Run GPU serving ? build / deploy / run (COMPUTE_DECOUPLED_SERVING M6/M7)
2
3       The cloud-serving compute path: a control plane (`CloudRunCompute`,
`backend/compute/cloud_run.py`)
4       dispatches a Cloud Run **Job** execution running this image's worker on an **L4**, then
**bucket-polls**
5       a done-marker for completion (D16). Scale-to-zero (D7) ? no warm pool.
6
7       > **Status:** the dispatch shim + image are **landed + mock-tested** (M6). The **real
build + push +
```

> run on an L4** is the **M7** owner step, within the **$100/?10k** budget (D17) ? confirm cmd + hourly
> cost first, **delete every resource after** ([[gcp-vm-always-delete]]). CI does **not** build this image.

0. Prerequisites (owner)
- A GCP project with billing on + the Cloud Run, Artifact Registry, and Cloud Build APIs enabled.
- GPU quota for L4 (`GPUS_ALL_REGIONS` ? 1) in the chosen region (asia-south1, else Singapore ? see
  `deploy/gcp/README.md`; L4 is often stocked-out, fall back per that runbook).
- The WASB weights staged in the bucket (WASB-C3 unresolved ? owner-gated): `gs://<bucket>/models/wasb_badminton_best.pth.tar`.

1. Build + push the image (Artifact Registry)
```bash
PROJECT=<gcp-project>; REPO=rally
# L4 GPU is region-limited ? use asia-southeast1 (Singapore), NOT asia-south1 (Mumbai).
REGION=asia-southeast1
gcloud artifacts repositories create $REPO --repository-format=docker --location=$REGION 2>/dev/null || true
# `gcloud builds submit` has NO -f flag, so a non-root Dockerfile needs a build config:
gcloud builds submit --config=deploy/cloudrun/cloudbuild.yaml --project $PROJECT .
```
(A CUDA + micromamba image is large; the first build is ~8?10 min. Budget-watch per D17.)

2. Create the Cloud Run **Job** (GPU, scale-to-zero)
The proven shape (first real M7 deploy, 2026-07-03): weights staged in a **same-region bucket**
mounted read-only via GCS FUSE (no weights in the image ? WASB-C3; no init-fetch step needed).
```bash
gcloud run jobs create rally-worker \
  --image $REGION-docker.pkg.dev/$PROJECT/$REPO/$IMG:latest \
  --region $REGION --gpu 1 --gpu-type nvidia-l4 --no-gpu-zonal-redundancy \
  --cpu 8 --memory 32Gi \
  --max-retries 0 --task-timeout 3600 \
  --set-env-vars WASB_WEIGHTS=/gcs/models/wasb_badminton_best.pth.tar \
  --add-volume name=serving,type=cloud-storage,bucket=$SERVING_BUCKET \
  --add-volume-mount volume=serving,mount-path=/gcs
```
- **`--no-gpu-zonal-redundancy` is REQUIRED on the CLI**, not just a cost choice (it also halves
  the GPU rate ? right for batch jobs): without the flag `gcloud` asks an interactive Y/n which,
  in a non-interactive shell, **hangs forever with zero output and nothing created**. An empty
  `gcloud` that "did nothing" ? suspect a hidden prompt.
- 8 vCPU / 32 GiB: decode + x264 are CPU-side (NVDEC is a deferred D16 knob), so CPU headroom is
  what keeps the paid GPU busy. `/tmp` is **RAM-backed** ? the pulled input video lives in memory,
  so size `--memory` for the largest expected upload (a 5.8 GB GoPro file ? 6 GiB of tmpfs).

3. Wire the dispatcher (control plane)
The control plane runs with the **cloud-serving** profile (`DEPLOYMENT_PROFILE=cloud-serving` ?
`compute_target=cloud_run`). `get_compute_backend(config)` then builds `CloudRunCompute` from env:
```bash
export DEPLOYMENT_PROFILE=cloud-serving
export CLOUD_RUN_JOB=rally-worker  CLOUD_RUN_REGION=$REGION GOOGLE_CLOUD_PROJECT=$PROJECT
```

```
# A `worker infer` (or the API job path) with a gs:// video now DISPATCHES to the Job instead of
# running in-process; the container runs the SAME worker INLINE (compute_target forced local_gpu).
python -m backend.worker infer --video-uri gs://<bucket>/videos/clip.mp4 --out-uri gs://<bucket> --segmenter wasb_hybrid
```
The AI handoff needs its key IN THE JOB env (`GEMINI_API_KEY` ? mount a Secret Manager secret
on the job), or run fully-local with `--set indexing.skip_ai_handoff=true` (candidate windows
emitted as rallies ? the eval-parity mode). With neither, the segmenter used to bail to a silent
0-rally "success" **after** the paid GPU pass; the cloud-serving startup checks now fail this at
boot (`AI_HANDOFF_KEY_MISSING`, rc 2) before the video is even pulled.

3a. Control-plane SERVICE deploy ? required flags for reliable async jobs

The API server (`/api/process`, `/api/compile` ? #329) runs its **job worker IN-PROCESS**: the
request returns `202` and a background thread drains the job (GPU detection dispatches to the Job;
the reel stitch runs on the control plane itself). On Cloud Run that background thread only makes
progress while the instance has CPU, so a scale-to-zero **service** deploy MUST set:
```bash
gcloud run deploy rally-control-plane --image <image> --region $REGION \
  --no-cpu-throttling \        # CPU stays allocated between requests ? the drain thread finishes
  --min-instances 1 \          # one always-warm instance drains reliably (cheap CPU; NOT the D7 GPU pool)
  --cpu 2 --memory 4Gi --port 8080 --allow-unauthenticated \
  --set-env-vars DEPLOYMENT_PROFILE=cloud-serving,CLOUD_RUN_JOB=rally-worker,CLOUD_RUN_REGION=$REGION,GOOGLE_CLOUD_PROJECT=$PROJECT
```
Without `--no-cpu-throttling`, a `/api/compile` returns 202 and then **stalls** ? the reel never
finishes and startup crash-recovery would mark it terminal. `min-instances 1` does NOT contradict
D7 (that was the ~$500/mo warm **GPU** pool); this is a ~1-vCPU CPU instance. Defense-in-depth is in
the code: `Database.recover_stale_jobs()` **re-queues** a compile interrupted by a restart (idempotent
stitch) rather than failing it, so even an unlucky scale-down mid-stitch self-heals on the next drain.
Reserve `--allow-unauthenticated` for the pre-CF-auth smoke test only; lock ingress to the CF proxy
(M9) before inviting testers.

4. M7 acceptance (the owner runlog)
Capture a `DEPLOY_REPORT_cloud-serving_<date>.md` under
`docs/COMPUTE_DECOUPLED_SERVING/runlogs/` (mirrors the truly-local template): the exact commands,
rally counts, `gpu_active_seconds`/`wall_seconds`/`bytes_out`, a reel smoke-test (`ffmpeg -f null -`
exit 0 + a 5-frame contact sheet), gotchas, and the **$0 teardown audit**.

Gotchas
- The dispatched container **must run inline** ? `CloudRunCompute` forces `compute_target=local_gpu`
  + `indexing.detector_impl=native` in the per-execution args so the Job never re-dispatches itself.
- Completion is a **bucket marker** (`<out_uri>/_dispatch/<token>.done`), not the Cloud Run API status

96 ? so it reuses the storage layer + `execution_policy.poll_until` and survives a control-plane restart.
97 - A **failed** Job (validation rc 2 / segmenter rc 3) now writes a **failure marker** carrying the
98 real non-zero rc, so the dispatcher's poll terminates **FAST + accurately** (no 30-min hang on a bad
99 video). Only a **hung/crashed** container (no marker at all) makes the bounded poll time out
100 (`ExecutionPolicy.max_wait_sec`); the worker CLI maps that to a clean **rc 4**, and the API path
101 surfaces it as a `cloud dispatch error` job failure (not a traceback).
102 - libx264 encode (parity, D16) + NVDEC decode-only; the image keeps ffmpeg's default libx264.
103 - **`GOOGLE_CLOUD_PROJECT` is required** for the real client ? `GoogleCloudRunJobClient.run_job`
104 fails fast with a clear `RuntimeError` if it is unset/empty (it must resolve a real `job_path`;
105 an unset project would otherwise build a bogus `projects/None/...` name). Export it (step 3) before
106 dispatching. The guard runs before the `google-cloud-run` import, so it is exercised in CI even
107 though the SDK isn't a CI/test dependency.
108 - **Hand-rolled `jobs execute --args` with `gs://` URIs:** gcloud's default comma-split plus its
109 alternate-delimiter syntax both bite ? repeated `--set` tokens trip the comma parser, and a
110 `^:^` delimiter splits the `:` inside `gs://`. Use a delimiter that appears nowhere in the args,
111 e.g. `--args='^@@^-v@@infer@@--video-uri=gs://?'`. (The dispatcher path doesn't have this
112 problem ? prefer it; this is for one-off diagnostics.) Container-arg overrides must include the
113 dispatcher's two forced `--set`s: `deployment.compute_target=local_gpu`,
114 `indexing.detector_impl=native`.
115 - **Git Bash (MSYS) on Windows mangles Unix-looking values** before gcloud sees them:
116 `--add-volume-mount ?mount-path=/gcs` became `C:/Program Files/Git/gcs`, and
117 `--command=/bin/bash` became `C:/Program Files/Git/usr/bin/bash` (the container then exec-fails).
118 Run gcloud commands with such flags from PowerShell (`MSYS_NO_PATHCONV=1` breaks gcloud's own
119 wrapper). PowerShell has its own trap: quote any flag value containing commas.
120 - **First execution after an image push** spends minutes in "Container image import in progress"
121 (visible in `executions describe status.conditions`, NOT in logs) ? that's progress, not a hang;
122 later executions reuse the imported image.
123

5. user (2026-07-04T01:48:41.383Z)

1 # Compute-Decoupled Rally Serving ? one pure core, profile-selected compute + storage
2
3 > **Status:** ? **M0 APPROVED** (owner, 2026-06-21) ? **IN PROGRESS (build M1+)** · **Owner:** Avi · **Created:** 2026-06-21 · **Last updated:** 2026-06-21
4 > **Lifecycle:** `DRAFT ? IN PROGRESS ? DONE ? archived` (move to `docs/archives/past_projects/` when DONE)
5 > **Tracking anchors:** §7 progress tracker is the source of truth; pointer in memory `current-state` + `docs/NEXT_STEPS.md` once work starts.
6 > **Relation to existing docs:** the productionization successor to the archived `DECOUPLED_COMPUTE` (M3/M4); realizes the `ComputeBackend`/`StorageBackend` registries from [PLATFORM_ARCHITECTURE.md](../PLATFORM_ARCHITECTURE.md) §3; the **truly-local** profile is the L0 tier of [VIDEO_LOCALITY_MODEL.md](../VIDEO_LOCALITY_MODEL.md) and the `LocalDesktop` backend of [DESKTOP_APP_PLAN.md](../DESKTOP_APP_PLAN.md).
7 > **Naming:** this was the "Cloud Run + GPU serving subproject" (ADR-012). Renamed **compute-decoupled serving** because **truly-local is a co-equal, first-class profile**, not an

afterthought.

> **Honesty note:** claims tagged `[verified]` (checked against code by the design workflow `wf_56274ff0`), `[design]` (proposed), `[researched]`.

> **? Northstar (D14):** a **mobile, app-like** flow ? point at a **Google Drive** video ? **fully cloud-native** run ? the stitched, **ready-to-share** reel lands **back in Drive, next to the source**. Two design promises around it: a user can **switch local?cloud anytime** (D13, a headline feature), and execution is **never a surprise** ? auto-detect *suggests*, the user *confirms* (D15).

0. TL;DR

The rally engine is a **pure function of `(input bytes + config) ? outputs`** in three deterministic stages ? **DETECT ? WINDOW ? STITCH** ? that **no deployment profile may branch on**. A **`ComputeBackend` (deployment profile)** decides *where* the core runs (a local process vs a Cloud Run GPU job); a **`StorageBackend`** decides *where bytes come from* (local FS / USB / mounted Google Drive / bucket). **One startup config** picks both. We ship **truly-local** (local GPU + in-process stitch, zero cloud) and **cloud-serving** (upload `<2GB` / gdrive / bucket ? a Cloud Run GPU job runs detect **and** stitch, so stitch's compute is metered), plus **cpu-mock** for CI. The single most important caveat: detection is **byte-identical only on the same GPU** (cross-GPU is sub-pixel-different, per the 2026-06-20 deploy report) ? so the **parity guarantee is enforced at the WINDOW + STITCH layer**, not detector-CSV bytes.

1. Problem & goal

- **Why now:** DECOUPLED_COMPUTE proved compute *portability* (M0?M2 + M3.5 on a GCP L4) and is archived; its deferred **M3 (serving endpoint + per-video billing) + M4 (scale-to-zero + cost guard)** become this project. The owner needs **both** a hosted service *and* a truly-local mode, with the **core (detect+stitch) unaffected** by which one runs.
- **The two user-facing outcomes** (owner, 2026-06-21):
  - **(a) Hosted service** ? operate on small **local uploads (`<2GB`)** *and* load from **gdrive / a bucket**, spinning up **Cloud Run for both rally segmentation AND stitching** (stitching is compute-intensive ? run it where its cost is accounted for).
  - **(b) Truly-local** ? on a local machine with videos on **local drives/USB + a local GPU**, run inference **and** stitching **completely locally**.
- **"Good" looks like:** the same `video ? rallies ? reel` core gives identical windows + stitched ranges whether it runs on a laptop or Cloud Run; switching is a **config/startup choice**, never a code branch in the core.

2. Decisions locked

#	Decision	Source	Implication
D1	The core is a **pure 3-stage function** (DETECT/WINDOW/STITCH); **no profile branch inside it**.	design `[verified]`	All profile difference lives in config + the two registries.
D2	**Two registries:** `ComputeBackend` (where) selected by `deployment.profile` + `compute_target`; `StorageBackend` (what-bytes) by `input_backend`/`storage.backend`.	ADR-014	Realizes PLATFORM_ARCHITECTURE's `local|hosted|byo_worker`.
D3	**Profiles shipped:** `truly-local`, `cloud-serving`, `cpu-mock` (CI); `byo_worker` reserved/deferred.	ADR-014	Enum/seam reserved so BYO isn't precluded.
D4	In **cloud-serving, STITCH runs on Cloud Run** (co-located with detect) so CPU/wall + egress are metered into the per-job cost ledger.	owner (a)	Stitch-on-laptop would hide real cost.
D5	**Parity enforced at WINDOW+STITCH** (byte-exact for a fixed trajectory); **detect** is byte-exact only same-GPU.	deploy report 2026-06-20	Cross-GPU asserted at the rally/window level, never CSV bytes.
D6	**Default-OFF, smallest-safe-first, one PR per milestone**; any core-caller change (e.g. configurable output base) ships behind a flag + a Launch Report.	[[launch-report-convention]]	Zero-regression discipline.
D7	**Cloud Run cold-start: SCALE-TO-ZERO** ? accept the ~1?3 min cold-start (it's an async job: upload?poll?reel, amortized over the multi-minute job). **No warm pool** (that ~$500/mo idle would break per-video billing).	owner 2026-06-21	M6/M7. Revisit a warm lane only on a UX complaint.
D8	**Pricebook = a versioned DB table**; cost computed in **USD** (matches GCP		

billing = one source of truth), **displayed in INR** at a configurable FX rate (Bangalore market); re-versioned when GCP prices change. | owner 2026-06-21 | M8. |
36 | D9 | **Edge auth = Cloudflare Access** (reuse the site's existing CF). Lock the Cloud Run **ingress to the CF proxy** + **verify the `Cf-Access` JWT signature** (so a direct hit to the Cloud Run URL can't spoof the identity header). | owner 2026-06-21 | M9. One identity system. |
37 | D10 | **Free-tier "data-for-reels" trade:** free reels in exchange for **training rights** (uploaded footage + derived trajectories/labels); **paid tier = no-train** (privacy is the paid feature). **Carve-outs:** train ONLY on attested **ADULT** footage (minor footage ? reel yes, training pool **NO** ? DPDP/GDPR need verifiable parental consent); train on **DERIVED** data + **auto-delete raw**; **ToS counsel-reviewed**; live training-on-uploads **gated to M9** (public signup). Truly-local needs no DPA. | owner 2026-06-21 | M9 + D12; ties [[paper-coauthor-aim]] / PERSONALIZATION flywheel. |
38 | D11 | **Quality floor: Gemini handoff ON** for cloud-serving until the owned model clears the #172 ship-gate (e.g. ?0.70 multi-court) ? then flip via a **Launch Report**; the owned model runs in **shadow/eval** meanwhile. (Gemini = serving/inference ONLY, never a training source ? CLAUDE.md guardrail.) | owner 2026-06-21 | M7 + D12. |
39 | D12 | **Data-flywheel harness is IN SCOPE (owner add, Q5):** capture the (consented, adult) uploaded video + the Gemini reel/rally output ? **reliably store** in the per-video workspace/bucket ? **run shadow evals** (owned-vs-Gemini, dual-eval-set) ? **promote good ones to the golden TRAINING set** (human-reviewed labels) so the owned model climbs toward the D11 floor (then Gemini flips off). | owner 2026-06-21 | new **M11**; ties D10 (consent) + `data_pipeline/` + the eval/experiment harness. |
40 | D13 | **Profile is user-switchable ANYTIME ? a headline FEATURE.** The same user runs **local today, cloud tomorrow, back to local** ? it's just a config/startup choice; the pure core gives **equivalent** output either way. **Advertise it** ("your machine *or* our cloud ? your call, switch anytime"). | owner 2026-06-21 | Honest caveat: equivalent at the **rally/window** level, not bit-identical across GPUs (D5); a single job runs in one profile, switching is *between* jobs. |
41 | D14 | **NORTHSTAR ? operate toward this:** a **mobile, app-like** flow ? point at a **Google Drive** video ? **fully cloud-native** run ? the stitched, **ready-to-share** reel lands **back in Drive, next to the source**. | owner 2026-06-21 | Implies a **`gdrive-api` (OAuth) StorageBackend** (read source + write reel back) ? distinct from the local **mount** backend; **resolves §8 #7**, new **M12**. The mobile UI is the khelsutra track; the engine must support it (async jobs + status + Drive round-trip / signed URL). Net-new scope (OAuth + Drive write); not necessarily v1, but the design must not preclude it (the `StorageBackend` ABC accommodates it). |
42 | D15 | **Auto-detect SUGGESTS, the user CONFIRMS ? execution is NEVER a surprise.** A GPU owner may still choose cloud; **cloud (= spend) is never entered silently**; the active profile is always explicit + surfaced. | owner 2026-06-21 | Amends §4.3 (auto-detect = a proposed default, not an auto-decision). |
43 | D16 | **M6/M7 implementation SHAPE + the cheap §8 picks (owner 2026-06-21):** (a) **Cloud Run JOBS** (not Services-with-GPU) for the detect+stitch unit, **bucket-poll for completion** (reuses `JobWaiting`/`claim_due_job` verbatim, fits async upload?poll?reel + scale-to-zero D7); (b) **stitch ENCODE = libx264** (protects the D5 byte-determinism/parity guarantee + the L4 parity test) ? **NVDEC for DECODE only** (a speed lever that never touches output bytes); **NVENC encode is DEFERRED behind a config knob**, revisit only if M7 cost data justifies relaxing parity (§8 Q8); (c) **input cap = hard-reject `<2GB`** for v1 (a config cap so raising to 4GB later is one line; matches the shipped M2 chunked-upload threshold ? §8 Q9). | owner 2026-06-21 | M6/M7. Resolves §8 Q2 (co-located, = D4), Q8, Q9, Q10 (dir name confirmed). |
44 | D17 | **Real cloud verification BUDGET = $100 / ?10k cap (owner 2026-06-21):** the owner authorized actual GCP spend for due-diligence + production-grade testing/verification of M5?M7 (real L4 run, the M5 runlog, the M7 DEPLOY_REPORT), **provided the cap is respected** ? confirm exact cmd + hourly cost before each spend, prefer the smallest viable run + cheapest-adequate GPU, and **DELETE every resource after** ([[gcp-vm-always-delete]]). Removes the M6/M7 "spend" gate within the envelope; the calibrated pricebook (M8) + counsel ToS (M9) + OAuth app (M12) stay owner/counsel-side. | owner 2026-06-21 | Unblocks the real M5/M7 runs (not just the code) within the cap. |
45
46 ## 3. Foundation ? evolution / ADR lineage ? *trace the idea end-to-end*
47 This project is the latest step in a long decision chain; each ADR contributed a piece and a **subtlety that carries forward**:
48
49 | ADR / doc | Contributed | Subtlety carried into this project |
50 |---|---|---|

51 | **ADR-005** | Permissive licensing (MIT/Apache-2.0/BSD only) | The engine + any served model + the **container image** (ffmpeg, fonts, weights) stay commercial-clean. |
52 | **ADR-008** | Owned-model topology; **train==serve featurizer single-sourcing** | The "core unaffected across profiles" is the same parity discipline, extended from train/serve to **local/cloud**. |
53 | **ADR-009** | KhelSutra Guru; **local-first delivery** | The **truly-local profile** is the direct continuation ? privacy/offline self-host stays first-class. |
54 | **ADR-010** | DECOUPLED: DO?GCP pivot | The cloud-compute origin (GPU + co-located bucket). |
55 | **ADR-011** | DECOUPLED scope = M0?M2+M3.5; **deferred M3 (serving) + M4 (billing/cost-guard)**; overrode the *"rent nothing / not a service"* posture | **This project realizes M3/M4.** Carries §06's owner-gated gates: **DPA/consent for non-owner uploads, collector `md5` keying, WASB-C3**. |
56 | **ADR-012** | GPU-native; TPU deferred; **Cloud Run+GPU scale-to-zero = serving model**; **NVDEC** named as the decode lever | The serving substrate + the 7-step seed scope this expands. |
57 | **ADR-013** | Drop DO; **GCP sole compute stack**; $200 DO credits ? non-compute only | The provider for cloud-serving; DO Spaces remains only a possible S3 *storage* target. |
58 | **? ADR-014 (new)** | **Compute-decoupled serving:** one pure core, profile-selected compute+storage; stitch-where-metered | This doc. **Refines** ADR-011/012, does **not** supersede them. |
59 | PLATFORM_ARCHITECTURE.md | The `ComputeBackend`/`StorageBackend` registries | The abstraction this realizes in code. |
60 | VIDEO_LOCALITY_MODEL.md / DESKTOP_APP_PLAN.md | L0 fully-local tier / `LocalDesktop` backend | The truly-local profile = these, productionized. |
61 | 07-COMPUTE-ABSTRACTION (archived) | `DeviceContext`/`DeviceProfile` (designed) | WASB hardcodes `device='cuda'` with **no CPU fallback** ? a portability gap M4 closes. |
62 | DEPLOY_REPORT_GCP_2026-06-20 (archived) | First real L4 run; **cross-GPU sub-pixel parity finding** | Why D5 (parity at window level, not CSV bytes). |
63
64 ## 4. Design / architecture
65 See [`architecture.svg`](architecture.svg) for the picture. **The core never changes; the box around it does.**
66
67 ### 4.1 The core contract (must stay byte/behaviour-identical) `[verified]`
68 1. **DETECT (GPU):** `DetectorRunner.run_predict(video_path, output_dir, video_id) ? CSV[Frame,Visibility,X,Y]` (`backend/pipeline/detectors/base.py:164`). Three interchangeable impls ? `WasbRunner` (WSL), `NativeWasbRunner` (Linux GPU), `StubDetectorRunner` (CPU mock) ? emit the **identical CSV schema**. The trajectory CSV is the stable artifact boundary.
69 2. **WINDOW (CPU pure fn):** `trajectory_to_action_windows(points, fps, frame_width, ?) ? windows` (`tracknet_runner.py:281`). A shared `@staticmethod`, zero GPU/IO/randomness ? same trajectory + config = byte-identical candidate windows. Used by every runner *and* the eval/regression stack verbatim.
70 3. **STITCH (CPU/ffmpeg):** `VideoCompiler.compile_highlights(raw_video, segments, out_name) ? reel` (`stitcher/compiler.py:303`). Pure FS+subprocess (merge ? ffprobe ? watermark ? per-clip libx264 trim ? concat). Deterministic for a given input + ranges + watermark config.
71
72 **Non-goal (the trap to avoid):** *no* code branch inside detect/window/stitch keyed on profile ? the same discipline the experiment harness already enforces (a disabled cue is byte-identical to CONTROL).
73
74 ### 4.2 The deployment profiles
75 | Profile | Compute | Storage | Orchestration | When |
76 |---|---|---|---|---|
77 | **truly-local** | `compute_target=local_gpu`; `detector_impl=native` (Linux) / `auto` (WSL); stitch **in-process** | `local` (or `gdrive` = mounted Drive) | `python -m backend.main` or `worker infer` CLI; existing async job worker | Self-host / offline / privacy / dev-on-a-GPU-box ? owner (b) |
78 | **cloud-serving** | `compute_target=cloud_run`; `detector_impl=native` (L4, cuda); **detect AND stitch on the same Cloud Run job** | `gcs` (or gdrive mount / assembled upload); per-video workspace; reels via signed URL | Cloud Run control plane enqueues ? Cloud Run GPU job pulls/runs/pushes; `WAIT_AND_RESUME` reschedules the control-plane job; scale-to-zero | Hosted SaaS ? owner (a) |
79 | **cpu-mock** | `compute_target=cpu_mock`; `detector_impl=stub` (synth/replay) | `local`/in-memory fakes | `run_all_tests.py` / pytest | CI, regression, profile-parity, GPU-free

dev |
80 | *byo_worker* (deferred) | tenant GPU via outbound pull agent | tenant's own bucket (signed URLs) | long-poll pull; same job table scoped by tenant | Privacy-sensitive enterprise ? **DEFERRED**, enum/seam reserved |
81
82 ### 4.3 Profile selection (the "startup/setup config") `[design]`
83 ONE new top-level `deployment` section on `AppConfig` + preset application + env auto-detect in `load_config`:
84 - `deployment.profile ? {truly-local | cloud-serving | cpu-mock}` (empty = today's manual knobs, **fully backward-compatible**).
85 - Selecting a profile applies a **coherent preset bundle** of *existing* knobs that today drift independently: **INPUT** (new `input_backend`/`input_root`, parallel to the output-side `StorageConfig`), **COMPUTE** (new `compute_target` enum locking `detector_impl` + `skip_ai_handoff` + workspace strategy), **STORAGE** (`storage.backend`/`workspace_root`/`single_folder_per_video`, all already exist).
86 - **Precedence:** built-in defaults ? profile preset ? `config.json` ? env (`RALLY_DETECTOR_IMPL`, `WASB_*`, `DEPLOYMENT_PROFILE`, ?) ? worker `--set k=v`. Every knob stays individually overridable.
87 - **Env auto-detect SUGGESTS, the user CONFIRMS (D15 ? no surprise execution):** the env gives a *proposed* default (`K_SERVICE` ? cloud-serving; `CI=true` ? cpu-mock; else `torch.cuda.is_available()` ? truly-local), but the user **explicitly confirms or overrides** it ? a GPU owner may still pick cloud, and **cloud (= spend) is NEVER entered silently**. The active profile is always surfaced (logged/shown). **Per-profile startup checks fail/warn fast** (cloud-serving without GCS creds/GPU ? loud error before any job).
88 - The worker (`cmd_infer`/`cmd_train`) is **already profile-agnostic** (reads `config.storage`, accepts `--config`/`--set`) ? no worker rewrite.
89
90 ### 4.4 Compute placement ? and *why stitch runs on Cloud Run* `[design]`
91 DETECT on GPU everywhere; WINDOW CPU-pure everywhere; STITCH is real CPU work. In **cloud-serving, stitch is co-located in the Cloud Run job** so `gpu_active_seconds` (detect) + `wall_seconds` (detect+stitch) + `bytes_out` (reel egress) land on **one metered invocation** ? the per-job cost ledger. Running stitch on a laptop would leave that cost **invisible/unattributed** ? exactly what the owner wants to avoid. (Stitch stays synchronous in the GPU job for the common one-reel case; a queued CPU-only stitch fallback only if compiles get bursty ? the job table + `claim_due_job` already supports both.)
92
93 ### 4.5 Reuse map ? most of the seams already exist `[verified]`
94 **Exists:** the 3-stage core; `_resolve_runner` (the single compute seam); `StubDetectorRunner` (`replay_csv` = byte-exact $0 anchor); `StorageBackend` ABC + 4 backends + `StorageRef.parse_uri`; the worker CLI (fetch/put, `--config`/`--set`); the async job control plane (`jobs` table, `claim_due_job` CAS); `ExecutionPolicy WAIT_AND_RESUME`/`JobWaiting` (carries to Cloud Run dispatch verbatim); the chunked-upload router (`<2GB`); `skip_ai_handoff` (LocalNoAI); storage fakes; golden fixtures + experiment harness. **Partial:** per-video workspace helper (default-OFF); `DeviceContext` (designed); edge-auth (v4 `user_id` columns exist). **Net-new:** the `deployment` section; `input_backend`; configurable output base (kill hardcoded `output/`); Cloud Run dispatch shim + container image; cost ledger + pricebook; edge-auth header extraction; startup env detection.
95
96 ### 4.6 The data-flywheel harness (D12 / M11) `[design]`
97 Gemini-on (D11) ships good reels now; this harness is **how we earn the right to flip Gemini off**. On a **consented, adult** cloud-serving job (the D10 trade), the pipeline additionally:
98 1. **Captures** the source upload + the Gemini reel + the rally output (intervals/report) into the per-video workspace (`workspaces/<video_id>/`), instead of the no-consent auto-delete path.
99 2. **Reliably stores** them durably in the bucket (the consented-retention bucket; raw video kept only for the training pool, not the no-consent path).
100 3. **Runs shadow evals** ? the owned model scores the same video alongside Gemini (`backend/eval/experiment.compare` + the dual-eval-set, [[eval-dual-set-regression]]) ? tracks the owned-vs-Gemini gap per job (the live read on "is the floor reachable yet?").
101 4. **Promotes to golden** ? captured videos that pass review become **golden TRAINING rows** (human-reviewed labels via the annotation flow ? `data_pipeline/`), growing the corpus ? the owned model climbs to the D11 floor ? flip Gemini off via a Launch Report.
102 This is the consent?capture?eval?goldenize **flywheel** (the cloud realization of the PERSONALIZATION contribution model): every consented free reel makes the owned model better.

Reuses `data_pipeline/`, `backend/eval/`, and the M9 consent gate; **adults-only**, raw-retention gated by the D10 consent.
103
104 ## 6. Honest limits ? what this does NOT do / prove
105 - A green CI suite proves **plumbing + determinism (window+stitch) + profile-parity** for $0 ? it does **NOT** prove field quality, nor that real-GPU detection is good (that's owner-side, real-video).
106 - **Cross-GPU detection is NOT byte-identical** ? parity is a window-level claim, not a CSV-byte claim.
107 - The cloud `0.1.0-l4` weights are **n=2 plumbing-grade** ? cloud-serving ships with **Gemini handoff ON** until the owned model clears a floor (open Q).
108 - This does not resolve **WASB-C3** (sharing a trained model) or the **DPA/consent** gate ? both remain owner/counsel-gated before public, non-owner serving.
109
110 ## 7. Deliverables & progress tracker ? **source of truth**
111 Legend: ? Todo · ? In progress · ? Done · ? Gated. **One small PR per row**, off `origin/master`, no stacking. Default-OFF where it touches core callers.
112
113 | ID | Deliverable | Depends | Gated? | Status | PR |
114 |----|-------------|---------|--------|--------|----|
115 | M0 | This design doc + project dir + `runlogs/` + ADR-014 | ? | owner sign-off | ? | **? APPROVED 2026-06-21** |
116 | M1 | `deployment.profile` + `compute_target` config + preset bundles + precedence/auto-detect; unit tests (no behaviour change, profile='' default) | M0 | safe | ? | [#279](https://github.com/Khelsutra/badminton-highlight-indexer/pull/279) |
117 | M2 | `input_backend`/`input_root`; wire main CLI/API input through `StorageRef.parse_uri` (local=identity); storage-fake tests | M1 | safe | ? | [#280](https://github.com/Khelsutra/badminton-highlight-indexer/pull/280) |
118 | M3 | Configurable output/scratch base ? kill hardcoded `output/` (`trajectory_hybrid:237,313`, `yolo_hybrid:407`, `gemini`); **DEFAULT stays `output/`**; golden + stub-E2E byte-identical | M1 | ? default-OFF + Launch Report on flip | ? | [#282](https://github.com/Khelsutra/badminton-highlight-indexer/pull/282) |
119 | M4 | `DeviceContext` + WASB **CPU-fallback**; unified healthcheck/device wiring | M1 | safe (cuda default unchanged) | ? | [#283](https://github.com/Khelsutra/badminton-highlight-indexer/pull/283) |
120 | M5 | **truly-local profile** end-to-end (preset + startup checks); first committed **runlog** (truly-local sample run) | M2,M3,M4 | owner real-GPU | ? | [#286](https://github.com/Khelsutra/badminton-highlight-indexer/pull/286) (preset+startup-checks+L4 parity ?; runlog ? owner GPU) |
121 | M6 | Cloud Run GPU **dispatch shim** (control plane ? job via storage ? re-claim) + **containerized worker image** (ffmpeg+CUDA+WASB+fonts); mocked-dispatch tests | M5 | ? owner (GCP spend) | ? | [#287](https://github.com/Khelsutra/badminton-highlight-indexer/pull/287) (shim+image+mocked ?; real build/run = M7 ? owner) |
122 | M7 | **cloud-serving profile** e2e on L4: upload`<2GB` + gdrive/bucket ? detect+stitch on Cloud Run ? reel via signed URL; **DEPLOY_REPORT** (posterity, sample runs + contact sheet) | M6 | ? owner (spend) | ? | ? |
123 | M8 | Per-job **cost ledger** (v6 migration: `owner_id, gpu_active_seconds, wall_seconds, bytes_out, cost_usd, cost_breakdown_json`) + **pricebook** + aggregation + `/api/.../cost` | M7 | ? owner (billing) | ? | [#288](https://github.com/Khelsutra/badminton-highlight-indexer/pull/288) (ledger+pricebook+/cost ? default-OFF; real prices + usage-emission ? owner/follow-up) |
124 | M9 | **Edge auth** (Cf-Access extraction) + per-tenant scoping on `user_id`; DPA/consent gate wiring | M7 | ? owner (privacy/counsel) | ? | [#290](https://github.com/Khelsutra/badminton-highlight-indexer/pull/290) (auth: Cf-Access JWT verify + owner_id tag, default-OFF ?; **+ exposure-safety boot posture** in `startup.py` ? edge-auth-on requires the `allowed_video_root` jail + CF team/AUD or the server refuses to start, see changelog; consent cols + ToS/DPA ? owner/counsel) |
125 | M10 | `byo_worker` / zero-infra-BYO ? **design-only, DEFERRED** | M8,M9 | ? explicit owner approval | ? | ? |
126 | **M11** | **Data-flywheel harness (D12):** on a consented adult job, **capture** the upload + Gemini reel/rally output ? **reliably store** under the per-video workspace ? **shadow-eval** owned-vs-Gemini (dual-eval-set / `experiment.compare`) ? **promote** good ones to the golden TRAINING set (human-reviewed labels via the annotation flow). Closes the loop so the owned model climbs to the D11 floor. Reuses `data_pipeline/` + `backend/eval/` + the consent gate (M9). | M7,M9 | ? owner (consent/privacy) | ? | **skeleton landed** (see changelog):

`backend/flywheel.py` + `FlywheelConfig` (default-OFF) + gated `_maybe_run_flywheel` hook, **consent faked-deny**; live activation ? owner/counsel (consent schema) + owned-model/golden data |
| **M12** | **`gdrive-api` (OAuth) StorageBackend (D14 Northstar):** cloud reads the source from + writes the stitched reel **back to the user's Google Drive next to the source** (per-user OAuth consent) ? the mobile/cloud-native Drive round-trip. Distinct from the local mount backend; slots into the `StorageBackend` ABC. | M7,M9 | ? owner (OAuth/consent) | ? | **backend landed** (see changelog): `backend/storage/gdrive_api.py` `GDriveApiStorage` + `gdrive-api://` scheme + ADC build; hermetic fake-Drive tests. **Live per-user OAuth app ? owner** |

Testing strategy is a first-class deliverable ? [`TESTING_STRATEGY.md`](TESTING_STRATEGY.md). **Run logs / sample runs for posterity** ? [`runlogs/`](runlogs/).

8. Open questions ? owner *(blocking-gates vs nice-to-knows)*
? The 5 gating questions are LOCKED (owner, 2026-06-21) ? see §2 D7?D12:
1. ? **Cold-start ? D7** (scale-to-zero, accept cold-start).
3. ? **Pricebook ? D8** (versioned DB table; USD internal / INR display).
4. ? **Edge auth ? D9** (Cloudflare Access; verify Cf-Access JWT, lock ingress to proxy).
5. ? **DPA/consent ? D10** (free-tier data-for-reels trade; adults-only; derived-data; counsel-gated to M9).
6. ? **Quality floor ? D11** + the **data-flywheel harness D12 / M11** (Gemini-ON until the owned model clears the floor; capture?store?eval?goldenize meanwhile).

? The remaining smaller picks are LOCKED (owner, 2026-06-21) ? see §2 D16:
2. ? **Stitch placement ? co-located** in the detect job (= D4; one metered unit; the job table supports a future CPU-split non-breaking, revisit only on bursty-compile metrics).
7. ? **RESOLVED ? D14:** cloud needs a **`gdrive-api` (OAuth) StorageBackend** (read the source from + write the reel back to the user's Drive) ? the Northstar's Drive round-trip ? distinct from the local **mount** backend (which stays for truly-local). New milestone **M12**.
8. ? **Codec ? D16:** **libx264 ENCODE** (protect parity) + **NVDEC DECODE-only** (speed, no output-byte change); NVENC-encode deferred behind a config knob.
9. ? **Input cap ? D16:** **hard-reject `<2GB`** for v1, as a config cap (raise to 4GB later in one line).
10. ? **Directory name ? `COMPUTE_DECOUPLED_SERVING/` CONFIRMED final** (already the name everywhere + in ADR-014); `CLOUD_RUN_GPU_SERVING/` dropped.

9. Definition of done
- [ ] M1?M4 land (config + input + output-base + device layer), each green on golden cross-OS + experiment no-regression.
- [ ] **Profile-parity test green** ? truly-local vs cloud-mock produce byte-identical windows + segment ranges.
- [ ] **truly-local** (M5) and **cloud-serving** (M7) each have a committed **runlog / DEPLOY_REPORT** with a real sample run.
- [ ] Cost ledger (M8) makes GPU + **stitch** + egress cost visible per job; edge auth (M9) gates multi-tenant.
- [ ] **Data-flywheel harness (M11)** captures?stores?shadow-evals?goldenizes consented adult jobs; the owned model is on a measured path to the D11 floor.
- [ ] ? flip Status DONE, archive to `docs/archives/past_projects/`.

10. References
ADR lineage: ADR-005/008/009/010/011/012/013 ? **ADR-014** (`docs/archives/decisions/DECISIONS.md`). **Strategy:** `PLATFORM_ARCHITECTURE.md`, `HOSTING_PLAN.md`, `VIDEO_LOCALITY_MODEL.md`, `DESKTOP_APP_PLAN.md`, `LOCAL_APPLIANCE_ARCHITECTURE.md` (khelsutra). **Archived predecessor:** `docs/archives/past_projects/DECOUPLED_COMPUTE/` (esp. `05-COST-ATTRIBUTION.md`, `07-COMPUTE-ABSTRACTION.md`, `06-RISKS-AND-OPEN-QUESTIONS.md`, `DEPLOY_REPORT_GCP_2026-06-20.md`). **Code anchors `[verified]`:** `backend/pipeline/detectors/base.py:164`, `tracknet_runner.py:281`, `stitcher/compiler.py:303`, `segmenters/trajectory_hybrid.py:65,237`, `backend/storage/*`, `backend/worker/__main__.py:88`, `backend/api/{uploads,job_worker}.py`, `backend/infrastructure/database.py`. **Design provenance:** workflow `wf_56274ff0` (4 code-mappers ? synthesis; 58 findings).

157 ---
158 ### Changelog
159 - 2026-06-21 ? created from the design workflow output; M0 in progress.
160 - 2026-06-21 ? **M0 APPROVED**; **M1 landed** ([#279](https://github.com/Khelsutra/badminton-highlight-indexer/pull/279)): `deployment.profile` + `compute_target` + preset bundles + precedence/auto-detect-as-suggestion (D15) on `AppConfig`. Pure/additive, `profile=''` default == today's behaviour (zero core change); presets touch only existing knobs (no M2 `input_backend` / M3 output-base over-reach). Suite 1068 green, cov 84.11%.
161 - 2026-06-21 ? **M2 landed** ([#280](https://github.com/Khelsutra/badminton-highlight-indexer/pull/280)): `input_backend`/`input_root` + URI-aware main API/CLI input via `resolve_input_uri`/`acquire_local_input` (local = identity passthrough, byte-for-byte unchanged; bucket/Drive fetched to scratch). Unsupported scheme ? clean 400; `local://` accepted on the resolved key; hosted-mode rejects non-local URIs; source URI kept for DB/report provenance. Suite 1105 green, cov 84.84%.
162 - 2026-06-21 ? **M3 landed** ([#282](https://github.com/Khelsutra/badminton-highlight-indexer/pull/282)): configurable output/scratch base via `backend/utils/paths.output_base(config)` ? the 3 segmenters' chunk/handoff/temp dirs root under `output.output_dir` (default `"output"`, byte-identical; default-OFF ? no Launch Report). Fixes the latent bug where scratch ignored `--output-dir`. Suite 1114 green, cov ~84.9%.
163 - 2026-06-21 ? **M4 landed** ([#283](https://github.com/Khelsutra/badminton-highlight-indexer/pull/283)): `DeviceContext` + native WASB **CPU-fallback** via a new `device="auto"` (cuda-if-available-else-cpu); explicit `"cuda"` stays strict (no silent slow-CPU downgrade), default unchanged. `"auto"` resolves before argv (never reaches `wasb_infer.py`); CUDA probe cached. Device-selection tested offline; real CPU-inference run owner/GPU-verified. Suite 1126 green, cov 84.96%. **? M1?M4 (the config/input/output/device layer) COMPLETE.**
164 - 2026-06-21 ? **M5+ batch authorized + design picks locked** (owner): the remaining §8 picks resolved ? **D16** (M6 = Cloud Run **Jobs** + bucket-poll; **libx264 encode + NVDEC decode-only**; **hard-reject `<2GB`** input cap; dir name confirmed) and **D17** (real cloud verification budget = **$100/?10k cap**, delete-after). Owner gave a blanket go to write+merge the **default-OFF code** for every remaining gated milestone (M5,M6,M8,M9-auth,M9-consent-cols,M11-plumbing,M12), one isolated PR each, reserving the gate for the real run/spend/calibration/counsel. Grounded by gate-map workflow `wf_4849400f`.
165 - 2026-06-21 ? **REAL M7 cloud GPU run ?** (owner-authorized, within the D17 cap): provisioned an **L4** (`g2-standard-4`, asia-southeast1-a; Mumbai stocked-out) ? native WASB on the L4 ? **22 rallies** on the smoke clip ? `gs://?/outputs/05e0e577?/{intervals.csv,report.json}`; VM DELETED ($0, ~$0.21). ? the no-Gemini cloud run needs `--set indexing.skip_ai_handoff=true` (the fully-local path emits WASB candidate windows directly ? the long-flagged "0 rallies" cold-start fix). The reel **stitch** validated locally (`VideoCompiler` ? 22-segment highlights, 85 MB). DEPLOY_REPORT runlog = the cloud-GPU dry-run guide (`docs/CLOUD_GPU_DRYRUN_GUIDE.md`).
166 - 2026-06-21 ? **M9-auth code landed** ([#290](https://github.com/Khelsutra/badminton-highlight-indexer/pull/290)): `backend/api/auth.py` ? Cloudflare Access **Cf-Access JWT verify** (RS256: sig+exp+iss+aud against the team JWKS; lazy `PyJWT[crypto]`; injectable JWKS ? fully-offline tests) + `resolve_tenant` ? tags `create_job(owner_id=?)`; `SecurityConfig.{edge_auth_enabled=False,cf_team_domain,cf_access_aud}` + CORS via `RALLY_CORS_ALLOW_ORIGINS`. **Default-OFF.** Security review (`wf_9b4c71d0`) found NO bypass; locked alg=none + HS256-confusion rejection. **? M9 ? ? the consent cols + ToS/DPA are owner/counsel; the consent migration is now v7 (M8 took v6).**
167 - 2026-06-21 ? **M8 code landed** ([#288](https://github.com/Khelsutra/badminton-highlight-indexer/pull/288)): per-job **cost ledger** ? `backend/billing.py` (pure `usage × pricebook` math, USD internal / INR display) + a **v6 migration** (NULLABLE cost columns on `jobs` + a versioned `pricebook` table seeded `placeholder-v0` at $0; additive-NULL, idempotent) + `record_job_cost`/`get_job_cost`/`get_video_cost` + `GET /api/jobs/{id}/cost` + `/api/videos/{id}/cost` + `BillingConfig` (**default-OFF**) + a gated `_maybe_record_job_cost` hook. Adversarial review (`wf_498b24af`) folded a **major silent-GPU-under-bill** (an unpriced `gpu_type` is now surfaced, not invisible $0) + the **honest no-op** caveat (usage emission is a follow-up ? cost 0 until wired) + breakdown reconciliation. Suite green, ruff+mypy clean. **? M8 ? ? owner inserts real GCP prices (a new pricebook version) + the worker usage-emission is a follow-up; placeholder ships $0 = no user-facing change. v6 is the shared bump (M9-consent cols extend it).**
168 - 2026-06-21 ? **M6 code landed** ([#287](https://github.com/Khelsutra/badminton-highlight-indexer/pull/287)): `backend/compute/` ComputeBackend registry ? `CloudRunCompute` dispatches a Cloud Run **Job** (injectable client) then **bucket-polls** a done-marker via `execution_policy.poll_until` + the storage layer (D16); `get_compute_backend` returns a backend

ONLY for `compute_target=cloud_run` (else `None` ? the worker's in-process path, **default-
OFF**). The dispatched container runs **inline** (forced `local_gpu`+native, never re-dispatch);
the worker writes a `--done-uri` completion marker on success.
`deploy/cloudrun/{Dockerfile,README}` = the L4 image (CUDA+ffmpeg+fonts+`wasb` conda env;
weights staged per WASB-C3) + the M7 runbook. Adversarial review (`wf_87b22fa4`) folded a
blocker (cloud `fetch` needs a dst ? reads silently returned `{}` on gcs/s3) + a poll-wait
test gap + `PolicyTimeout`?clean rc 4. Suite green, ruff+mypy clean. **? M6 ? ? the real
build/push/L4 run + DEPLOY_REPORT is M7 (owner, D17 budget).**
169   - 2026-06-21 ? **M5 code landed** ([#286](https://github.com/Khelsutra/badminton-
highlight-indexer/pull/286)): `backend/config/startup.py` ? per-profile fail/warn-fast startup
checks (cloud-serving + non-cloud storage = the one fatal coherence error; GPU/creds mismatches
= warnings, the runner healthcheck stays the authority), wired into the server
(`_enforce_startup_or_exit` ? exit 1) + the worker (`cmd_infer`/`cmd_train` ? rc 2), default-OFF
(probe gated behind an active profile ? the worker stays torch-free at `profile=""`). **L4
profile-parity test** (`tests/test_profile_parity.py`): same stub trajectory ? byte-identical
windows + segment ranges across truly-local / cpu-mock / no-profile + worker-vs-in-process.
Adversarial 4-lens review folded (`wf_34671ac1`). Suite green, ruff+mypy clean. **? M5 ? ? the
real-GPU truly-local runlog (L5) is owner-side** (template pre-staged in `runlogs/`).
170   - 2026-06-21 ? **M9 hardening: exposure-safety boot posture** (alpha-shareable "tunnel-
to-a-box" enabler): extended `backend/config/startup.py` with a **server-only exposure posture**
(new `include_exposure=True` from `_enforce_startup_or_exit`; the worker stays unaffected ? it
reads `--video-uri`, not untrusted tenant paths). Gated on `security.edge_auth_enabled`
(**default-OFF ? strict no-op**, byte-identical to today): when the owner flips edge auth ON to
expose the engine behind the CF-Access gate, three checks fire at **boot** ?
`EXPOSED_NO_VIDEO_ROOT` (**fatal**: `allowed_video_root` jail unset ? a tenant could read an
arbitrary local file), `EXPOSED_EDGE_AUTH_INCOMPLETE` (**fatal**: CF team-domain/AUD missing ?
lifts the current first-request 401 to boot), `EXPOSED_AUTH_EXTRA_MISSING` (**warn**:
`PyJWT[crypto]` not importable ? would 500 per request). So a half-configured exposure fails
fast instead of leaking/500-ing. +15 unit tests (default-OFF no-op, worker-ignores, both-fatals-
compose-with-profile-checks, server-exit, WARN-only-no-exit). Adversarial 3-lens review
(`wf_35b735a9`) confirmed default-OFF byte-identical + no bypass; folded a **whitespace-drift**
(aligned `auth.resolve_tenant` to `.strip()` team/AUD so "configured" means the same at boot +
at request time) + named the config fields in the edge-auth-incomplete message. Pairs with the
appliance **T3** network posture (raw `:8000` refused ? owner-run). Suite green, ruff+mypy
clean. **? backstops M9 ? the actual exposure flip + CF team/AUD/ingress + key rotation stay
owner-side. (Deferred to the PR-2 runbook: document that `upload_dir` must sit under
`allowed_video_root`, and that you MUST set `edge_auth_enabled=True` when exposing ? these
checks only fire then.)**
171   - 2026-06-22 ? **M11 skeleton landed** (data-flywheel harness, D12):
`backend/flywheel.py` (`capture_job` ? per-video `VideoWorkspace` via `backend.storage.put`;
`shadow_eval` ? `experiment.compare`/`compare_unlabeled`; `propose_promotion` ?
`promotion.promote_if_served_safe` with Gemini=control/owned=proposal; `run_for_job`
orchestrator) + `FlywheelConfig` (**default-OFF**) + a gated `_maybe_run_flywheel` job-
completion hook mirroring M8's cost hook. **TWO independent guards keep it INERT in
production:** `flywheel.enabled=False` (default) AND `consent_attested` is a **FAKED hard-deny**
(returns False for every job) ? so even enabled it captures **nothing** until the real consent
schema lands. This is the PLUMBING only; it reuses the real workspace/eval/promotion/storage
code so it works the day consent + the owned model + golden labels are wired. **NO DB
migration** (the skeleton persists no consent). +13 tests (default-OFF no-op, enabled-but-
consent-denied no-op, consented-capture writes the manifest, delegation of shadow-eval/promote,
the hook is best-effort). Suite green, ruff+mypy clean. **? M11 ? ? LIVE activation is
owner/counsel (the consent ToS/DPA + schema ? the consent columns) + owner/GPU (the owned model
+ golden corpus for shadow-eval/promotion). ? the future M9-consent migration is now `v9` (v7 =
`locale`, v8 = the compute-picker `compute` column ? the schema ladder drifts as siblings land;
check `PRAGMA user_version` before authoring).**
172   - 2026-06-22 ? **M12 backend landed** (`gdrive-api://` OAuth/API StorageBackend, D14
Northstar): `backend/storage/gdrive_api.py` `GDriveApiStorage(StorageBackend)` ? the headless
Drive-API round-trip (read source + write reel back), distinct from the mount-only
`gdrive://` (left untouched). Path-keyed `gdrive-api://<path>` resolves each component ? a Drive
file id via `files().list`; implements `fetch_to_local` (`get_media().execute()` bytes), `put`
(folder-creating `files().create` + a `_media_body` seam), `exists`/`stat`/`list`/`delete`.
Wired the scheme into `ref.py` (`_KNOWN` + the path-keyed branch), `__init__._backend_class`,
and the `StorageConfig.backend`/`input_backend` Literals + a `gdrive_api` settings dict (read
via a **hyphen?underscore** lookup in `get_storage`). **All Google libs lazy-imported** (`import
backend.storage` never pulls them); a `service` is **injectable** ? fully-offline tests via an

in-memory fake Drive (round-trip + scheme + registry + the underscore-settings lookup); `_build_service` (ADC over the Drive scope) raises a clear `storage-gdrive-api`-extra error when libs/creds are absent. **NO DB migration.** Adversarial 2-lens review (`wf_ae8c9b0f`). Suite green (1466), ruff+mypy clean. **? M12 ? ? the LIVE per-user interactive OAuth app (consumer "connect your Drive": client id/secret + refresh tokens) is owner-gated; the backend ships the service-account/ADC build + the fake. Reads no secrets from config (creds via ADC only).**

6. assistant (2026-07-04T01:48:46.180Z)

Both DEPLOY_REPORT and ALPHA_LAUNCH_READINESS need reading. The DEPLOY_REPORT already exists.

7. user (2026-07-04T01:48:48.230Z)

```
# DEPLOY_REPORT ? cloud-serving (M7) first real L4 run + GPU optimization sweep

> **Date:** 2026-07-04 · **Owner-authorized** GCP spend (D17, $300 envelope) · deleted every
> resource after (`[[gcp-vm-always-delete]]`; §5 residual audit = $0 of ours).
> **Video:** `MahadevpuraSingles.MP4` (owned golden, 13,418 frames). **Job:** Cloud Run L4
> (Ada sm_89, 24GB) · 8 vCPU / 32GB · `--no-gpu-zonal-redundancy` · scale-to-zero.
> **Mode:** fully-local (`skip_ai_handoff=true`) ? the number is the **owned WASB path only, zero Gemini**.

## 1. M7 infra proven
- Worker image builds + pushes (`asia-southeast1/rally`); the M6 `CloudRunCompute` shim dispatches the
  L4 Job and bucket-polls the done-marker; WASB weights via a GCS-FUSE read-only mount (no weights in
  image ? WASB-C3); `intervals.csv` + `report.json` land in the bucket.
- Owned model runs on the L4: `native exec wasb_infer.py`, 13,418 trajectory points, 48 windows, 0 Gemini.

## 2. GPU optimization ? measured
**Headline: the L4 is CPU-FEED-bound, not GPU-bound.** GPU utilization sat **<1%** on every config ?
a GPU that idle is *waiting*, not computing. So kernel/GPU levers do nothing; the wins (and the
remaining headroom) are all on the CPU feed.

| # | Config | Phase-2 wall | vs baseline | GPU util | Parity (rally windows) |
|---|--------|-------------:|------------:|:--------:|------------------------|
| 0 | cu113, batch 8 (baseline) | 727.9s | ? | <1% | reference |
| 1 | torch 2.2.2+cu118, batch 8 | 722.1s | ?0.8% (noise) | <1% | intervals byte-identical; trajectory differs (6780 vs 6777 pts) |
| 2 | cu113, **batch 64 + prefetch 4** | **506.1s** | **?30.5% (1.44×)** | ~1% | **byte-identical** ? (6777 pts) |
| 3 | cu113, batch 128 + prefetch 6 | OOM | ? | ? | exceeds 24GB VRAM (`CUDA OOM, 9 GiB`) |

### Dispositions
- **? SHIP ? batch/prefetch tuning (config #2):** 30.5% parity-safe. Batch 64 is at the L4 VRAM
  ceiling (128 OOMs), so it's the max viable value, not arbitrary. ? PR `claude/wasb-gpu-batching`.
- **? DISCARD ? torch cu113?cu118 arch bump (config #1):** 0.8% = noise (JIT was never the wall,
  since the GPU was idle regardless), *and* it perturbs trajectory numerics (a byte-parity risk).
  Empirically dead for this workload.
- **? DON'T ? bigger GPU:** idle at <1% ? an A100/H100 would also idle. Pure waste (D8).
- **? OPT-IN ONLY (never default) ? FP16 / TF32 / cudnn.benchmark / NVDEC:** each changes detector
  numerics ? breaks the D5/D16 byte-parity guarantee (verified reasoning in the lever
```

workflow).

3. Root cause & the remaining ~5× (scoped follow-up)
Streaming forces `DataLoader(num_workers=0)` (the in-memory `SequentialFrameStore` + `read_image`
shim can't pickle to worker processes), so decode?PIL?ToTensor runs on **one core**, inline with the
GPU; sliding windows re-transform each frame ~`frames_in`×. At batch 64 the GPU is *still* ~1% ?
batching only removed the per-batch round-trip/Python overhead; the ~500s floor is the single-threaded
feed. **The remaining 3?5× is a parallel-feed change**, not a config or a bigger machine:
- **Design:** partition the video into contiguous chunks, one DataLoader worker per chunk (each opens
  its own `VideoCapture`, seeks to its chunk start, reads forward ? sequential within a chunk,
  parallel across chunks); collate reorders. Parity-safe (same frames, same transform, same order).
- **Alternative (rejected for Cloud Run):** the existing disk-frames path already does `num_workers>0`,
  but materializing 13,418 PNGs into the RAM-backed `/tmp` (~27GB) OOMs the 32GB box ? which is *why*
  streaming exists. A real scratch disk (GCE VM, not Cloud Run) would unlock it.
- Only after the feed is parallel does a **higher-vCPU config** pay off (more cores to feed the GPU).

4. Cost ledger (D17)
- Builds: 4 × ~$0.16 (cu113 base reuse, cu118, batch) ? $0.5
- Runs: baseline + cu118 + batch64 + batch128(OOM, short) + probes ? ~$2?3 of L4 time
- Idle: **$0** (scale-to-zero; GPU bills only during execution)
- **Total well under the $300 / D17 envelope.**

5. Teardown ? residual audit ($0 of ours)
Deleted after all measurements: jobs `rally-worker`, `rally-worker-cu118`, `rally-worker-batch`,
`rally-probe`; the `asia-southeast1/rally` image repo; the `khelsutra-rally-serving` bucket; the
`gemini-api-key` secret. Post-teardown audit:
```
run jobs      ? Listed 0 items.
run services  ? Listed 0 items.
GCE VMs       ? Listed 0 items.
rally repo    ? NOT_FOUND
serving bucket? 404 not found
KEPT (owner's): gs://khelsutra-rally-corpus/, gs://khelsutra-rally-nightly/
```
(The `..._cloudbuild` bucket is Cloud Build's auto-managed source cache ? system-owned, not ours.)
No GPU/compute of ours remains; idle cost = $0.

8. user (2026-07-04T01:48:48.637Z)

Alpha Launch Readiness Plan

> **Status:** `DRAFT` - cross-repo execution tracker; **Owner:** @avidullu; **Created:** 2026-07-03; **Last updated:** 2026-07-04
> **Lifecycle:** `DRAFT -> IN PROGRESS -> DONE -> archived`
> **Tracking anchors:** Section 7 is the progress tracker and source of truth; indexed in `docs/README.md` and `docs/DOC_STATUS.md`.
> **Relation to existing docs:** consolidates the launch-relevant pending items from `NEXT_STEPS.md`, `ROADMAP.md`, `docs/data_pipeline/GOLDEN_VIDEOS.md`, `R2_EVAL_METRIC_DESIGN.md`, `COMPUTE_DECOUPLED_SERVING/`, and the sibling `khelsutra` Studio docs.

> **Honesty note:** `[verified]` means checked against the local synced repos on 2026-07-03. `[analysis]` means derived from a local command run on that snapshot. This is not legal, financial, or launch approval.

---

0. TL;DR

The corpus is now large enough to move from "grow labels first" into alpha-readiness work, but the 15-video headline is not itself a launch gate pass. `[verified]` The golden manifest has 15 badminton videos; `[analysis]` A2-A9 made the local manifest/path, full-fusion/audio, decision, and shadow-training surfaces explicit; `[analysis]` A4/A5 refreshed the n=15 nightly and heuristic floors; `[analysis]` A8/A9 still reject served Gate A default promotion and owned-model serving claims on the current evidence.

The first controlled alpha should be a hosted, authenticated product loop: upload a video, process it, select rallies, compile, and download a reel. The owned-model/commercial rally-detection claim remains a separate NO-GO until the ship-gate target is ratified and the WASB checkpoint provenance is resolved.

1. Problem & goal

The previous launch blockers were spread across roadmap docs and mixed together three different questions:

- **Can an invited tester use the product without SSH?**
- **Can the 15-video golden corpus now support honest quality gates?**
- **Can we claim or ship an owned/commercial rally detector?**

This doc separates those into executable workstreams. The goal is to reach an invite-only alpha where the product is usable end-to-end, the quality claims are honest and reproducible, and the remaining model/legal gates are explicit rather than accidentally implied by launch activity.

2. Decisions locked

#	Decision	Source / date	Implication
D1	**Product alpha and owned-model launch are separate decisions.**	`NEXT_STEPS.md` NO-GO banner, 2026-06-30; verified 2026-07-03	We can launch a controlled hosted product loop without claiming the owned model is commercially cleared.
D2	**First alpha must be page-driven, not SSH/CLI-driven.**	`NEXT_STEPS.md` P0 UI-driven/shareable item; verified 2026-07-03	Hosted service, edge auth, upload, job polling, compile, and download are launch-critical.
D3	**The 15-video corpus is now a measurement asset, not a free launch pass.**	`[verified]` `docs/data_pipeline/GOLDEN_VIDEOS.md`; `[analysis]` local eval runs, 2026-07-03	Rebase/portability, feature completeness, baseline refresh, and gate ratification come before quality claims.
D4	**Do not promote served Gate A by default from the current n=15 evidence.**	`[analysis]` `backend.eval.serve_contrast` run, 2026-07-04	Proposal-side lift does not survive served-windowing safety; keep it as an opt-in/analysis lever until owner ratification.
D5	**P10/P11 corpus promotion and train/re-eval are alpha-plus unless owner reopens D13.**	sibling `khelsutra/docs/STUDIO_MEDIA_LIFECYCLE.md`; verified 2026-07-03	First alpha can use manual/offline corpus promotion; automated flywheel follows after serving and gates are stable.
D6	**Gen-0 n=15 shadow training is non-serving.**	`[analysis]` `training.gen0.harness` run, 2026-07-04	Learned mean beats the heuristic floor, but the paired sign test is not significant; #172 and WASB C3 remain hard blockers.

3. Verified snapshot, 2026-07-03

All five Git checkouts were fast-forwarded before analysis:

Repo	Branch	Synced HEAD

`badminton-highlight-indexer`	`master`	`e4914d1`
`khelsutra`	`master`	`b0f70a5`
`rally-annotator`	`main`	`ef9e088`
`sports-data-collector`	`master`	`bfc014f`
`sports-obsreport`	`main`	`a0c7252`

Corpus/eval facts:

- `[verified]` `docs/data_pipeline/GOLDEN_VIDEOS.md` says `output/human_lovo_manifest.json` is the ingested golden corpus at `n = 15`.
- `[analysis]` The local manifest had stale absolute paths. A temporary rebased manifest found all 15 trajectories and labels in this workspace.
- `[analysis]` A7 generated full shuttle/person fusion schema for all 15 videos and A8 audio augmentation populated audio features for all 15.
- `[analysis]` `python -m backend.eval.fusion_compare --features-dir output` now runs over all 15 and measures person-fusion `Delta F1 = +0.008`; the paired CI crosses zero, so person fusion is not promotable by itself.
- `[analysis]` `python -m backend.eval.fusion_audio --features-dir output --compare` measures audio `Delta F1 = +0.018`; the paired CI crosses zero, so audio is a shadow/suggestive cue, not a default-flip.
- `[analysis]` `python -m backend.eval.serve_contrast --manifest output\human_lovo_manifest.json` measured proposal Gate A mean `Delta F1 +0.0287`, but served mean `Delta F1 -0.0328`; not served-safe.
- `[analysis]` `python -m training.gen0.harness --manifest output\human_lovo_manifest.json --floor eval_baselines\heuristic_lovo_n15.json --version 0.1.1-n15-shadow.20260704 --out artifacts\rally_tal_gen0` produced learned LOVO mean F1 `0.551`, floor passed versus heuristic `0.446`, but sign test `9/15`, `p = 0.3036`, `ship=False`.
- `[analysis]` `python scripts/nightly_regression.py --manifest <rebased-n15> --features-dir output --no-report` reported default/milestone tolF1 `0.3636` vs baseline-off tolF1 `0.1382`, but marked the frozen baseline stale due to config/corpus changes.
- `[analysis]` Before A6, `python -m backend.eval.ablation --features-dir output --granularity group` failed at import due to a circular import between `backend/eval/ablation.py` and `backend/eval/ablation_pdf.py`; A6 fixes the module entrypoint and keeps full-fusion ablation scoped to the 6 videos with full schema until A7.

4. Launch strategy

4.1 Alpha lane: make the product usable

The alpha lane is successful when an invited tester can open the app, authenticate, upload a video, watch the job progress, pick rallies, compile a reel, and download it. This lane should avoid model-quality promises beyond the measured state and should default to the most reliable serving path.

4.2 Quality lane: make the 15-video corpus gate-ready

The quality lane is successful when the 15-video corpus is portable, reproducible, feature-complete, and backed by refreshed baselines. Only then should R2 tolF1, Gen-0/TemporalMaxer, or other model/promotion decisions be ratified.

4.3 Flywheel lane: make tester feedback useful

The flywheel lane is successful when user corrections can flow into the collector/vault path with consent/provenance and a visible re-eval delta. This is valuable for alpha learning, but it should not block the first controlled alpha if manual promotion can bridge the gap.

5. Risks and guardrails

Risk	Why it matters	Guardrail
Launch activity implies model launch approval	The engine docs explicitly say rally-detection product launch is NO-GO until ship target and WASB C3 clear	Keep all alpha copy scoped to invite-only processing, not owned-model claims
Stale local manifests create false confidence	Manifest-driven tools skipped or failed until paths were rebased	Make the corpus manifest portable and add a path/fixture smoke

check |

15-video corpus is only partially feature-complete	Fusion/person/audio eval is still n=6 today	Regenerate or upconvert full-fusion features for all 15 before fusion decisions
Baseline refresh accidentally rubber-stamps a regression	Nightly runner correctly reported stale baselines	Refresh baselines only after reviewer sign-off and record the command/output
P10/P11 expands scope before hosted alpha is usable	Corpus flywheel is valuable but can consume the alpha launch window	Keep P10/P11 behind explicit owner reopen unless alpha lane is already stable
WASB checkpoint provenance remains unresolved	Blocks commercial sharing of a trained model regardless of F1	Treat C3 as a hard model-release gate; do not hide it inside product launch tasks

6. Honest limits

Completing this doc's alpha lane does **not** prove the owned model is ready, that WASB weights are commercially clean, that the product can serve unbounded public traffic, that multi-sport is ready, or that every tester correction automatically improves the model.

Completing the quality lane does **not** by itself deploy a service. It only makes the measurement surface honest enough for a launch decision.

7. Deliverables & progress tracker

Legend: TODO = not started, IN PROGRESS = actively being worked, DONE = shipped/verified, GATED = blocked on owner/legal/platform decision. One PR per row unless the row explicitly names a cross-repo batch.

ID	Deliverable	Repo / owner	Depends on	Gated?	Status	PR
A0	Create this alpha readiness tracker and index it in docs	`badminton-highlight-indexer`	-	No	DONE	#451
A1	Stand up alpha serving endpoint: Cloud Run/GPU or approved host, health check, edge auth ON, upload -> process -> jobs -> compile -> download verified	`khelsutra` + `badminton-highlight-indexer`	A0	Owner spend / CF config	TODO	-
A2	Make the n=15 golden manifest portable/reproducible; add a path/corpus smoke check that fails loudly when labels or trajectories are missing	`badminton-highlight-indexer`	A0	No	DONE	#452
A3	Promote the 15-video source-video/MD5 records into the collector/vault path; reconcile collector's six-video source manifest with the 15-video eval corpus	`sports-data-collector` + vault	A2	Owner upload/storage scope	DONE	collector #62
A4	Refresh the n=15 nightly regression baseline intentionally and record the exact command/output; keep stale-baseline warnings until reviewed	`badminton-highlight-indexer`	A2	Reviewer sign-off	DONE	#453
A5	Recompute the heuristic n=15 floor and replace the stale `heuristic_lovo_n6` floor for future Gen-0 comparisons	`badminton-highlight-indexer`	A2	No	DONE	#454
A6	Fix `backend.eval.ablation` CLI circular import, then run the R2/tolF1 ablation on the n=15 corpus	`badminton-highlight-indexer`	A2	No	DONE	#455
A7	Regenerate or upconvert full-fusion feature JSONs for the newer 9 videos so fusion/person/audio analyses run over all 15	`badminton-highlight-indexer`	A2	GPU/data availability	DONE	#458
A8	Re-run `fusion_compare`, group ablation, and served contrast after A7; explicitly promote/reject person fusion, Gate A served-default, and any other R-series default flips	`badminton-highlight-indexer`	A6, A7	Reviewer sign-off	DONE	#458
A9	Run Gen-0/TemporalMaxer shadow training against the refreshed n=15 floor; produce a non-serving artifact bundle and gate verdict	`badminton-highlight-indexer`	A5, A8	Owner approval for M0.4 scope	DONE	#458
A10	Decide and record the alpha ship-gate target: metric, corpus slice, minimum threshold, significance rule, and what claims are allowed	Owner + `badminton-highlight-indexer`	A4, A5, A8	Owner decision	GATED	-
A11	Resolve or explicitly defer WASB C3 provenance for alpha messaging; if deferred, document "no commercial model sharing" copy guardrail	Owner/legal	A10	Legal/provenance	GATED	-

| A12 | Reopen or defer P10/P11 corpus flywheel: `PATCH /api/segments`, collector promotion, `train`/`reeval` job, delta-F1 surface, and alignment with

`COMPUTE_DECOUPLED_SERVING` M11 | `khelsutra` + `badminton-highlight-indexer` + collector | A1, A3 | Owner D13 reopen (`khelsutra/docs/STUDIO_MEDIA_LIFECYCLE.md`) | GATED | - |
| A13 | Alpha launch review: runbook, privacy/consent copy, observability/job-failure triage, known limits, rollback/disable steps, and owner GO/NO-GO | Cross-repo | A1, A4, A10, A11 | Owner launch decision | TODO | - |

8. Concrete next PR sequence

1. **Docs PR:** land this tracker.
2. **Corpus portability PR:** rebase/portable manifest tooling plus missing-file smoke test.
3. **Hosted alpha PR/batch:** health/auth/upload/process/job/compile/download verification and runbook updates.
4. **Baseline PR:** intentional n=15 nightly baseline refresh and heuristic floor refresh.
5. **Ablation/fusion PRs:** fix ablation import, complete 15-video features, rerun promotion/rejection decisions.
6. **Launch decision PR/docs update:** record ship-gate target, C3 status, alpha copy limits, and launch checklist.

A2 smoke command:

```powershell
python -m backend.eval.golden_manifest --manifest output/human_lovo_manifest.json --min-count 15
python -m backend.eval.golden_manifest --manifest output/human_lovo_manifest.json --min-count 15 --write output/human_lovo_manifest.local.json
```

The checker resolves stale absolute paths against the repo's sibling `Annotation Setup` folders and `output/`, then exits non-zero if any required `trajectory` or `labels` file is absent.

A4 nightly baseline refresh record:

```powershell
python scripts\nightly_regression.py --manifest output\human_lovo_manifest.json --features-dir output --update-baseline
```

Result:

```text
INFO baseline snapshot written to C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\eval_baselines\nightly_baseline.json
INFO   default: n=15 tolF1=0.364 f1@0.5=0.472 merge_rate=0.135
INFO   milestone: n=15 tolF1=0.364 f1@0.5=0.472 merge_rate=0.135
INFO   baseline_off: n=15 tolF1=0.138 f1@0.5=0.237 merge_rate=0.638
```

Pass-mode proof:

```powershell
python scripts\nightly_regression.py --manifest output\human_lovo_manifest.json --features-dir output --no-report
```

Result: `PASS`, exit code `0`, baseline snapshot `2026-07-03`, default/milestone `tol_f1 = 0.3636`, `f1@0.5 = 0.4723`, `merge_rate = 0.1350`; baseline-off `tol_f1 = 0.1382`, `f1@0.5 = 0.2366`, `merge_rate = 0.6380`.

A5 heuristic floor refresh record:

```powershell
python -m backend.eval.calibrate_local --manifest output\human_lovo_manifest.json
```

--metric f1 --floor-out eval_baselines\heuristic_lovo_n15.json
```

Result:

```text
baseline (untuned) mean held-out f1: 0.237
calibrated mean held-out f1:        0.446 +/- 0.239  (gap +0.000)
```

The committed floor is `eval_baselines/heuristic_lovo_n15.json`: `n = 15`, `mean_f1 = 0.4456`, `std = 0.2390`, `metric = f1`, `iou = 0.5`. Every leave-one-video-out fold selected `(0.05, 1.0, 2.0, 0)`.

A6 ablation/R2 record:

```powershell
python -m backend.eval.ablation --features-dir output --granularity group
python -m backend.eval.rally_seg_eval --manifest output\human_lovo_manifest.json --features-dir output --preset served --max-window-duration 0 --inactivity-end 0 --vs served --vs-max-window-duration 6 --vs-inactivity-end 1.5 --json-out output\a6_r2_ablation.json
```

Result: the ablation CLI now runs, but full-fusion feature ablation still reports `n=6` because only the original six feature JSONs carry the full fusion schema; A7 must regenerate/upconvert the newer nine before fusion/person/audio promotion decisions. The n=15 R2/tolF1 contrast is positive for the served milestone versus baseline-off: baseline/off `tolF1 = 0.138`, served `tolF1 = 0.364`, `Delta tolF1 = +0.225` with CI `[+0.165, +0.283]`, sign `14/0`, `p = 0.000`; `Delta F1@0.5 = +0.236` with CI `[+0.161, +0.313]`; merge rate improves from `0.638` to `0.135`. Net over-segmentation guard passes (`1.276 -> 0.378`), while the strict segment-count rule remains blocked by increased splits, so R2/tolF1 is the supported gate for the baseline-off to served milestone contrast rather than a blanket fusion/default-flip approval.

A7 full-fusion/audio feature refresh record:

```powershell
python -m backend.eval.fusion_golden --manifest output\human_lovo_manifest.json --out-dir output --source-manifest ..\sports-data-collector\data\golden_source_videos.json --missing-fusion-only --smallest-first --max-gpu-temp-c 87
python -m backend.eval.fusion_audio --manifest output\human_lovo_manifest.json --features-dir output --source-manifest ..\sports-data-collector\data\golden_source_videos.json --augment
```

Result: the GPU pass completed without a thermal abort after fans were set to max performance. Full-fusion schema is now present for `15/15` feature JSONs, and audio features are present for `15/15`. The large regenerated clips completed as follows: `mahadevpura_1` `1` window / `0` positives / `10` GT, `GX020094` `99` / `30` / `40`, `GX030094` `79` / `21` / `41`, `GX010141` `26` / `11` / `29`, `GX010137` `56` / `31` / `34`, `Boxhill_Doubles` `59` / `12` / `43`, `mahadevpura_2` `14` / `2` / `40`, `GX010128` `51` / `15` / `52`, `Badminton_BXH_2` `29` / `1` / `44`.

A3 source-manifest follow-up: `sports-data-collector` PR #62 is merged. The authoritative collector manifest now records `13/15` present local source/proxy paths and two Boxhill run-log-only eval inputs: `GX010141_1080p.mp4` (`daaf8af198640516d14b217f50445d99`) and `GX010137_1080p.mp4` (`74eedbc35d739c231344b8929df4b4a7`). The F-drive 4K originals are preserved there only as `original_*` provenance, not as eval join targets, because the golden trajectories/labels are 1920-wide. Freshly re-extracting those two Boxhill full-fusion files requires recovering the exact 1080p eval inputs or adding a coordinate-normalization fix; do not substitute the 4K originals for person-cue extraction.

A8 fusion/default-decision record:

```powershell

```
python -m backend.eval.fusion_compare --features-dir output
python -m backend.eval.fusion_audio --features-dir output --compare
python -m backend.eval.ablation --features-dir output --granularity group
python -m backend.eval.serve_contrast --manifest output\human_lovo_manifest.json
python -m backend.eval.rally_seg_eval --manifest output\human_lovo_manifest.json --features-dir output --preset served --max-window-duration 0 --inactivity-end 0 --vs served --vs-max-window-duration 6 --vs-inactivity-end 1.5 --json-out output\a6_r2_ablation.json
```

Result:

- Person fusion is **not promoted**: shuttle-only F1 `0.302`, shuttle+person F1 `0.311`, `Delta F1 +0.008`, paired CI `[-0.040, +0.055]`, sign `8W/6L`, `p = 0.791`.
- Audio is **shadow/suggestive, not promoted**: shuttle+person F1 `0.311`, shuttle+person+audio F1 `0.329`, `Delta F1 +0.018`, paired CI `[-0.017, +0.053]`, sign `10W/4L`, `p = 0.180`.
- Group ablation remains non-significant for cue promotion: audio `+0.018` CI `[-0.017, +0.053]`; shuttle `+0.013` CI `[-0.002, +0.029]`; gate_a `-0.001` CI `[-0.012, +0.008]`; person `-0.009` CI `[-0.045, +0.021]`.
- Gate A served default is **rejected**: proposal mean `Delta F1 +0.0287`, CI `[-0.0222, +0.0680]`; served mean `Delta F1 -0.0328`, CI `[-0.0799, +0.0052]`, `regresses=True`.
- R2/cap/inactivity remains the strong quality result: served candidate F1 `0.472` vs baseline-off `0.237`, `Delta F1 +0.236`, CI `[+0.161, +0.313]`; tolF1 `0.364` vs `0.138`, `Delta tolF1 +0.225`, CI `[+0.165, +0.283]`; merge rate `0.135` vs `0.638`. Net over-seg guard passes, strict per-video split-count guard still blocks blanket promotion language.

A9 Gen-0 shadow-training record:

```powershell
python -m training.gen0.harness --manifest output\human_lovo_manifest.json --floor eval_baselines\heuristic_lovo_n15.json --out artifacts\rally_tal_gen0 --version 0.1.1-n15-shadow.20260704
```

Result: current Gen-0 LogReg/TAL harness produced local bundle `artifacts\rally_tal_gen0\0.1.1-n15-shadow.20260704` with learned LOVO mean F1 `0.551` versus heuristic floor `0.4456`, so `floor_passed=True`. The gate remains **non-serving**: paired sign test `9/15`, `p = 0.3036`, `ship=False`; blockers are uncalibrated ship-gate target #172 and WASB checkpoint provenance C3. TemporalMaxer itself is not implemented in this branch; the evidence supports either closing A9 as a Gen-0 shadow baseline or opening a follow-up M0.4 scorer-swap PR.

9. Open questions

1. **Owner:** Is the first alpha allowed to use manual/offline corpus promotion, or should P10/P11 be reopened before any tester wave?
2. **Owner/platform:** Which hosted compute target is the next stand-up: Cloud Run GPU, GCE L4 VM, or another approved GPU host?
3. **Owner/legal:** What is the acceptable alpha wording while WASB C3 remains unresolved?
4. **Engineering:** Should the portable corpus manifest live as a generated artifact, a checked-in redacted manifest, or both?
5. **Engineering:** Should A9 close on the current Gen-0 non-serving bundle, or should the actual TemporalMaxer scorer swap be implemented before marking A9 done?

10. Definition of done

- [ ] Invited tester can complete the hosted record/upload -> reel download CUJ without SSH.
- [ ] Alpha service is gated behind the chosen auth posture and has a documented disable/rollback path.
- [x] n=15 corpus is portable enough that manifest-driven quality commands do not rely on stale local absolute paths.
- [x] n=15 nightly baseline and heuristic floor are refreshed with recorded commands.
- [x] R2/tolF1 gate is promoted or rejected with a recorded ablation result.

- [x] Fusion/person/Gate-A served-default decisions are explicitly promoted or rejected on the 15-video corpus.
- [ ] Ship-gate target and allowed alpha claims are recorded.
- [ ] WASB C3 status is recorded and reflected in alpha messaging.
- [ ] P10/P11 are either reopened with a committed sequence or explicitly deferred for alpha-plus.
- [ ] Owner records GO/NO-GO for the first alpha wave.

11. References

Internal docs:

- `docs/NEXT_STEPS.md`
- `docs/ROADMAP.md`
- `docs/data_pipeline/GOLDEN_VIDEOS.md`
- `docs/R2_EVAL_METRIC_DESIGN.md`
- `docs/RALLY_DETECTION_FIXES_PLAN.md`
- `docs/COMPUTE_DECOUPLED_SERVING/README.md`
- sibling `khelsutra/NEXT_STEPS.md`
- sibling `khelsutra/docs/STUDIO_MEDIA_LIFECYCLE.md`
- sibling `khelsutra/docs/PER_RALLY_SELECTION.md`
- sibling `sports-data-collector/data/golden_source_videos.json`

Local verification commands used for the 2026-07-03 snapshot:

```powershell
python -m backend.eval.fusion_compare --features-dir output
python -m backend.eval.serve_contrast --manifest <rebased-n15-manifest>
python -m training.gen0.harness --manifest <rebased-n15-manifest> --floor eval_baselines\heuristic_lovo_n15.json --version analysis-n15 --out <temp-out>
python scripts\nightly_regression.py --manifest <rebased-n15-manifest> --features-dir output --no-report
python -m backend.eval.ablation --features-dir output --granularity group
python -m backend.eval.rally_seg_eval --manifest output\human_lovo_manifest.json --features-dir output --preset served --max-window-duration 0 --inactivity-end 0 --vs served --vs-max-window-duration 6 --vs-inactivity-end 1.5 --json-out output\a6_r2_ablation.json
```

Changelog

- 2026-07-03 - Created from a synced five-repo review of `NEXT_STEPS`, `ROADMAP`, golden-corpus docs, local eval artifacts, and the current 15-video corpus state.
- 2026-07-04 - Recorded A7 full-fusion/audio regeneration, A8 n15 fusion/default decisions, and A9 Gen-0 n15 shadow-training gate verdict.
- 2026-07-04 - Marked A3/A7/A8/A9 complete after collector #62 and indexer #458 review/CI; clarified that the two Boxhill eval inputs are run-log-only 1080p hashes and that F-drive 4K originals are provenance-only.

9. assistant (2026-07-04T01:48:52.524Z)

I have everything needed. Let me check DEPLOYMENT_PROFILE code gating and confirm the PR #457 provenance of the deploy report.

10. user (2026-07-04T01:48:54.166Z)

tests\test_api_cloud_dispatch.py:1:"""API cloud-serving dispatch (COMPUTE_DECOUPLED_SERVING M6 ? the API path).
backend\config\startup.py:3:A deployment profile (``truly-local`` / ``cloud-serving`` / ``cpu-mock``) implies a coherent
backend\config\startup.py:7: * **cloud-serving with a non-cloud storage backend** ? a hard **error** (the one config bug we
backend\config\startup.py:44:# Storage backends that count as "a cloud bucket" for cloud-serving

```
coherence.
backend\config\startup.py:240:    elif profile == "cloud-serving":
backend\config\startup.py:247:                    f"profile 'cloud-serving' requires a cloud
storage backend "
backend\config\startup.py:264:        # the first real cloud-serving litmus, 2026-07-03). The
env cannot appear later in
backend\config\startup.py:274:                    f"profile 'cloud-serving' runs the {provider}
handoff "
backend\config\startup.py:286:                    "profile 'cloud-serving' runs on a GPU (L4)
but indexing.wasb.device='cpu' ? "
backend\config\startup.py:295:                    "profile 'cloud-serving' expects a GPU but no
CUDA GPU is visible to this process.",
backend\config\presets.py:4:config knobs, so the truly-local / cloud-serving / cpu-mock modes
don't drift apart one
backend\config\presets.py:13:(``config.json`` ``deployment.profile`` or the
``DEPLOYMENT_PROFILE`` env var); with no
backend\config\presets.py:27:PROFILES: tuple[str, ...] = ("truly-local", "cloud-serving", "cpu-
mock")
backend\config\presets.py:34:    "cloud-serving": "cloud_run",
backend\config\presets.py:51:    "cloud-serving": {
backend\config\presets.py:52:        "deployment": {"profile": "cloud-serving",
"compute_target": "cloud_run"},
backend\config\presets.py:123:      - ``K_SERVICE`` set (Cloud Run)        ? ``"cloud-serving"``
backend\config\presets.py:129:    imports torch; a CLI ``--detect`` or cloud-serving startup
that wants the GPU probe passes
backend\config\presets.py:134:        return "cloud-serving"
backend\config\models.py:116:    # prefetch ? byte-identical to the historical path). The cloud-
serving preset opts into
backend\config\models.py:438:    ``deployment.profile`` or the ``DEPLOYMENT_PROFILE`` env var);
auto-detect only ever
backend\config\models.py:446:    profile: Literal["", "truly-local", "cloud-serving", "cpu-
mock"] = ""
backend\config\loader.py:58:    An *unrecognised* ``DEPLOYMENT_PROFILE`` env value warns and
FALLS THROUGH to the file's
backend\config\loader.py:62:    env_profile = os.environ.get("DEPLOYMENT_PROFILE")
backend\config\loader.py:68:            "[CONFIG] unrecognized DEPLOYMENT_PROFILE env value %r
(valid: %s); ignoring it "
backend\config\loader.py:177:    # explicitly selected (DEPLOYMENT_PROFILE env or config.json
deployment.profile). With no
backend\config\loader.py:231:                "set deployment.profile or DEPLOYMENT_PROFILE to
apply (auto-detect never "
backend\worker\__main__.py:288:                "indexing.skip_ai_handoff=true; cloud-serving now
fails this at startup), native "
tests\test_compute_backend.py:420:    # cloud-serving startup checks require the AI-handoff key
(AI_HANDOFF_KEY_MISSING is fatal).
tests\test_compute_backend.py:424:        '{"deployment": {"profile": "cloud-serving"},
"storage": {"backend": "gcs"}}'
tests\test_config_deployment.py:56:    monkeypatch.delenv("DEPLOYMENT_PROFILE", raising=False)
tests\test_config_deployment.py:120:def test_cloud_serving_profile(tmp_path):
tests\test_config_deployment.py:121:    cfg = load_config(_write(tmp_path, {"deployment":
{"profile": "cloud-serving"}}))
tests\test_config_deployment.py:122:    assert cfg.deployment.profile == "cloud-serving"
tests\test_config_deployment.py:204:                "deployment": {"profile": "cloud-serving",
"compute_target": ""},
tests\test_config_deployment.py:208:    assert cfg.deployment.profile == "cloud-serving"
tests\test_config_deployment.py:216:    monkeypatch.setenv("DEPLOYMENT_PROFILE", "cpu-mock")
tests\test_config_deployment.py:225:    monkeypatch.setenv("DEPLOYMENT_PROFILE", "cpu-mock")
tests\test_config_deployment.py:226:    cfg = load_config(_write(tmp_path, {"deployment":
{"profile": "cloud-serving"}}))
tests\test_config_deployment.py:233:    monkeypatch.setenv("DEPLOYMENT_PROFILE", "   ")

[Showing results with pagination = limit: 40]
```

11. user (2026-07-04T01:48:57.587Z)

```
c5fc27c wasb: parity-safe GPU-feed batching (measured -30.5% on L4) + DEPLOY_REPORT (#457)
---
a9f5848 wasb: validate batch_size/prefetch_batches as positive at config time (#459)
20b2d5a wasb: validate batch_size/prefetch_batches as positive at config time
c5fc27c wasb: parity-safe GPU-feed batching (measured -30.5% on L4) + DEPLOY_REPORT (#457)
5fb55e9 fix(cloudrun): install [storage-gcs] so the cloud worker can read/write gs:// (#337)
950106b fix(fusion): W2 net-crossings gate is do-no-harm ? never empty a non-empty video (#188 follow-up) (#190)
a471745 feat(eval): Tier 1 (W2) ? net-crossings rally-state gate on W1 (?F1 +0.019, 6/0) (#188)
---GREP PR457---
docs/archives/past_projects/AUDIT_REPORT_khelsutra-guru_2026-07-02-COMPLETED-2026-07-02.md
docs/QUALITY_ITERATIONS.md
docs/R2_EVAL_METRIC_DESIGN.md
docs/RALLY_DETECTION_QUALITY_REPORT.md
docs/RALLY_QUALITY_RESEARCH.md
```

12. assistant (2026-07-04T01:49:02.918Z)

Confirmed: the DEPLOY_REPORT_cloud-serving_2026-07-04.md was committed in PR #457 (commit c5fc27c). Let me confirm the code gating details in startup.py and presets.

13. user (2026-07-04T01:49:04.932Z)

```
            "deployment": {"profile": "truly-local", "compute_target": "local_gpu"},
            # detector_impl stays "auto" (the runner picks WSL vs native by env on the GPU box).
            # Zero-cloud / privacy-first ? run fully local with no Gemini handoff.
            "indexing": {"skip_ai_handoff": True},
            "storage": {"backend": "local"},
        },
        "cloud-serving": {
            "deployment": {"profile": "cloud-serving", "compute_target": "cloud_run"},
            # Linux L4 (cuda); Gemini handoff ON until the owned model clears the #172 floor (D11).
            # wasb GPU-feed tuning: the parity defaults (batch 8, no prefetch) left the paid L4
            # <1% utilized on the first real serving run (2026-07-03) ? the pipeline was
            # CPU-feed-bound. Serving opts into the measured fast values; local/WSL unchanged.
            "indexing": {
                "detector_impl": "native",
                "skip_ai_handoff": False,
                "wasb": {"batch_size": 64, "prefetch_batches": 4},
            },
            "storage": {"backend": "gcs"},
        },
        "cpu-mock": {
            "deployment": {"profile": "cpu-mock", "compute_target": "cpu_mock"},
            # Deterministic CPU mock ? no GPU/WSL, no provider/API. The CI / regression profile.
            "indexing": {"detector_impl": "stub", "skip_ai_handoff": True},
            "storage": {"backend": "local"},
        },
    }


    def is_known_profile(profile: str) -> bool:
        """True for a non-empty, recognised profile name."""
        return bool(profile) and profile in PROFILE_PRESETS


    def preset_for(profile: str) -> dict[str, Any]:
        """Return a deep COPY of the preset bundle for ``profile`` (so callers can't mutate the
        module-level table), or ``{}`` for ``""`` / an unknown profile."""
```

```
    return copy.deepcopy(PROFILE_PRESETS.get(profile, {}))


def _truthy(val: Optional[str]) -> bool:
    """Common env-var truthiness (``1/true/yes/on``, case/space-insensitive)."""
    if val is None:
        return False
    return val.strip().lower() in {"1", "true", "yes", "on"}


def _cuda_available() -> bool:
    """Best-effort CUDA probe. torch is heavy + optional, so it is imported lazily here and
    ONLY when a caller opts into ``probe_cuda``; any import/runtime failure means 'no GPU'."""
    try:
        # Intentionally lazy + local: keeps the config-load path torch-free (torch is heavy
        # and optional). Only reached when a caller opts into probe_cuda.
        import torch

        return bool(torch.cuda.is_available())
    except Exception:
        return False


def probe_cuda_available() -> bool:
    """Public best-effort CUDA probe (lazy torch import; ``False`` on any failure).

    Exposed for the per-profile startup checks (``backend/config/startup.py``) so a caller
    that *wants* the GPU heads-up can pass the result in without the startup module importing
    torch itself. Best-effort by design: if torch lives only in a detector subprocess env
    (native WASB), this returns ``False`` in the main process ? which is why the startup checks
    treat a GPU mismatch as a WARNING (the runner healthcheck is the authority), never a hard
    error that could false-fail."""
    return _cuda_available()


def suggest_profile(
    env: Optional[Mapping[str, str]] = None, *, probe_cuda: bool = False
) -> str:
    """SUGGEST (never auto-apply ? D15) a deployment profile from the environment.

    Signals, in priority order:

      - ``K_SERVICE`` set (Cloud Run)        ? ``"cloud-serving"``
      - ``CI`` truthy                        ? ``"cpu-mock"``
```

14. user (2026-07-04T01:49:05.503Z)

```
                        "to CPU (much slower). If torch lives only in the detector subprocess env this "
                        "can be a false alarm; the runner's DeviceContext/healthcheck is the authority.",
                    )
                )

    elif profile == "cloud-serving":
        # The ONE fatal coherence check: a cloud profile that can't reach a bucket can't serve.
```

```
        if storage_backend not in _CLOUD_STORAGE:
            checks.append(
                StartupCheck(
                    LEVEL_ERROR,
                    "STORAGE_INCOHERENT",
                    f"profile 'cloud-serving' requires a cloud storage backend "
                    f"({'/'.join(_CLOUD_STORAGE)}), but storage.backend='{storage_backend}'. "
                    "A cloud job cannot read/write a local-only backend.",
                )
            )
        elif storage_backend == "gcs" and not _creds_discoverable(e):
            checks.append(
                StartupCheck(
                    LEVEL_WARN,
                    "CREDS_UNCONFIRMED",
                    "no GOOGLE_APPLICATION_CREDENTIALS / Cloud Run env detected; relying on Application "
                    "Default Credentials. The GCS backend will fail loudly at init if ADC can't resolve.",
                )
            )
        # The AI-handoff key: with skip_ai_handoff=False the segmenter demands
        # ${AI_PROVIDER}_API_KEY at rally time and bails to an EMPTY result when it is
        # missing ? a silent 0-rally "success" AFTER the GPU pass is paid for (observed on
        # the first real cloud-serving litmus, 2026-07-03). The env cannot appear later in
        # a running container, so this is a zero-false-positive boot-time fatal.
        skip_ai = bool(getattr(indexing, "skip_ai_handoff", False))
        provider = str(getattr(config, "ai_provider", "gemini") or "gemini")
        key_var = f"{provider.upper()}_API_KEY"
        if not skip_ai and not e.get(key_var):
            checks.append(
                StartupCheck(
                    LEVEL_ERROR,
                    "AI_HANDOFF_KEY_MISSING",
                    f"profile 'cloud-serving' runs the {provider} handoff "
                    f"(indexing.skip_ai_handoff=False) but {key_var} is not set ? the segmenter "
                    f"would bail to an empty result after the paid GPU pass. Mount the key "
                    f"(e.g. a Secret Manager secret) or set indexing.skip_ai_handoff=true for "
                    "the fully-local candidates-as-rallies mode.",
                )
            )
        if device == "cpu":
            checks.append(
                StartupCheck(
                    LEVEL_WARN,
                    "CLOUD_CPU_DEVICE",
                    "profile 'cloud-serving' runs on a GPU (L4) but indexing.wasb.device='cpu' ? "
                    "detection will not use the GPU.",
                )
            )
        elif cuda_available is False and device in ("cuda", "auto"):
            checks.append(
                StartupCheck(
                    LEVEL_WARN,
                    "CLOUD_CUDA_NOT_VISIBLE",
                    "profile 'cloud-serving' expects a GPU but no CUDA GPU is visible to this process.",
                )
```

```
                )

        elif profile == "cpu-mock":
```

15. assistant (2026-07-04T01:49:08.959Z)

I have all evidence needed. Compiling structured findings.

16. user (2026-07-04T01:50:12.532Z)

Structured output provided successfully